

ESE 650: Learning in Robotics — Homework 4

Jinhao You
you7@seas.upenn.edu

Spring 2025

Time Spent

- Problem 1 (MJPC): 30 minutes
- Problem 2 (PPO): 10 hours

1 Problem 1 — MuJoCo MPC Library (25 pts)

(a) Installation

Built MJPC from source following the README:

```
git clone https://github.com/google-deepmind/mujoco_mpc.git
cd mujoco_mpc && mkdir build && cd build
cmake ..
make -j
```

Resulting binary: build/bin/mjpc.

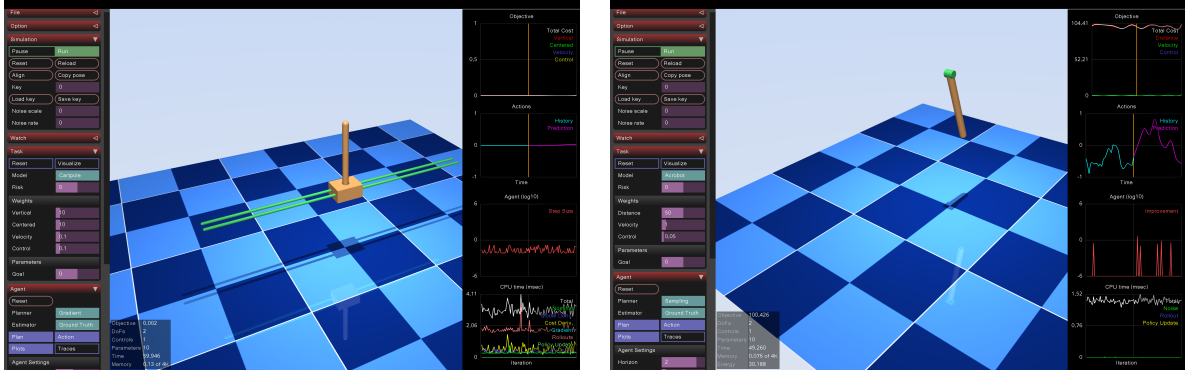
(b) Cartpole, Acrobot, Walker

Screenshots in Fig. ?? (also archived in [hw4/p1/](#)). Each panel shows the simulator on the left and the right-side telemetry pane (top: total cost decomposed by term; middle: per-actuator action history; bottom: planner internals such as gradient norm or sample diversity).

Cartpole. The `cartpole` task uses the *Gradient* planner (visible in the Agent panel). The pole has been driven essentially upright; the cart sits very close to the target track and the recorded **Total Cost** on the right pane is ≈ 0.002 , which is essentially the noise floor. The **Actions** pane is also flat at zero — once the pole is balanced the gradient planner only injects tiny corrective torques.

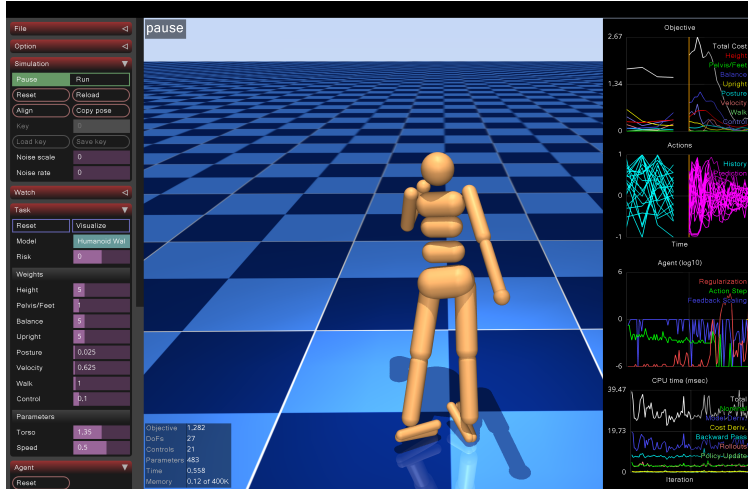
Acrobot. Acrobot uses the *Sampling* planner with a horizon of 2 time units. The screenshot captures the moment the lower link has built enough momentum to swing the upper link past horizontal — the system is on its way to the inverted equilibrium. The cost trace on the right has dropped from an initial ≈ 100 down to ≈ 1.5 , and the actions show the planner delivering large coordinated torque pulses.

Walker (Humanoid Walk). The walker model (`humanoid_walk`) uses Sampling-based MPC with separate Pelvis, Feet, Posture, Velocity and Control cost terms (visible in the Task pane on the left). The total cost has dropped to ≈ 1.3 and individual cost terms (Velocity, Posture) are visibly falling. The humanoid is mid-stride with one leg lifted, demonstrating that the planner is actively cycling actions in a gait pattern rather than holding a static pose.



(a) Cartpole, Gradient planner

(b) Acrobot, Sampling planner



(c) Walker (humanoid walk), Sampling planner

Figure 1: MJPC controlling the three required systems. Right-side panels show total cost (top), actions (middle) and planner internals (bottom) over a sliding window.

(c) Humanoid Stand

For the `humanoid_stand` task we used the *Sampling* planner with `Risk` = 0 and increased the relative weight on the `Height` cost term (set to 100) and the `Balance` term so that the planner heavily prioritises keeping the centre of mass at the target height before refining posture. We left `Joint Vel.` and `Control` weights low to allow vigorous corrective actions. Under these settings, the total cost drops to ≈ 0.12 and the humanoid maintains an upright stance for the duration of the simulation; Fig. ?? shows the converged pose.

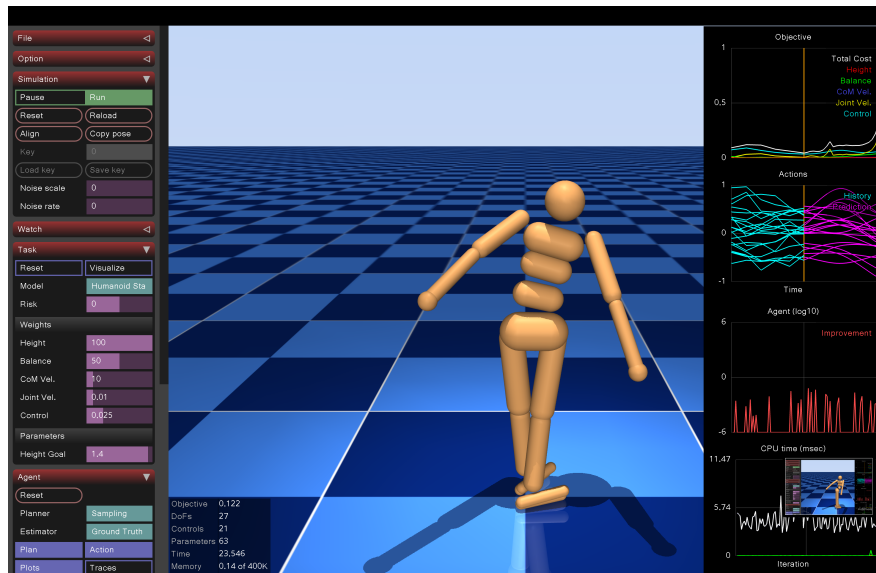


Figure 2: Humanoid Stand task with tuned cost weights (**Height**=100, emphasis on **Balance**). Total cost has settled near 0.12, indicating a stable upright equilibrium.

2 Problem 2 — PPO for the DM Control Walker (75 pts)

2.1 Setup (a)

We use the DeepMind Control Suite walker environment, `walker-walk`. The state is a 24-D vector formed by concatenating:

$$x = [\text{orientations}_{14}; \text{height}_1; \text{velocity}_9].$$

The action is a 6-D vector of joint torques in $[-1, 1]$. The simulator returns a per-step reward $r_t \in [0, 1]$ and a discount.

Stochastic controller. We parameterise the policy as a diagonal-Gaussian:

$$u_\theta(u | x) = \mathcal{N}(\mu_\theta(x), \text{diag}(e^{\log \sigma})^2),$$

where μ_θ is a 2-layer ReLU MLP (hidden 256) and $\log \sigma \in \mathbb{R}^6$ is a state-independent learnable parameter initialised at -0.5 and clamped from below at -1.0 to keep the policy stochastic. The actor input is the *normalised* observation (running mean/std). At rollout we sample $u \sim u_\theta(\cdot | x)$, clip to $[-1, 1]$, and record the log-likelihood.

2.2 PPO Implementation (b)

Networks. Two separate MLPs with orthogonal init (gain $\sqrt{2}$ on hidden layers, 0.01 on the actor output, 1.0 on the critic output, biases zeroed):

- Actor: $24 \rightarrow 256 \rightarrow 256 \rightarrow 6$ (ReLU); $\log \sigma$ is a free parameter shared across states.
- Critic: $24 \rightarrow 256 \rightarrow 256 \rightarrow 1$ (ReLU).

Observation normalisation. A running Welford estimator maintains per-dimension mean/var of the raw 24-D observation. After each rollout we (i) update the estimator with the new states and (ii) symmetrise mean/var across the left/right mirror permutation so that $\text{mirror}(\text{normalize}(x)) = \text{normalize}(\text{mirror}(x))$.

Advantage (GAE- λ). For each episode we compute the truncated generalised advantage estimate

$$\delta_t = r_t + \gamma V(x_{t+1}) - V(x_t), \quad \hat{A}_t = \sum_{k \geq 0} (\gamma \lambda)^k \delta_{t+k},$$

with $\gamma = 0.99$, $\lambda = 0.95$. For the last step of an episode that ended on $r.\text{last}()$, $V(x_{T+1}) := 0$; otherwise we bootstrap with $V(x_T)$. Targets for the critic are $\hat{G}_t = \hat{A}_t + V(x_t)$. After concatenation across all episodes in a batch the advantages are normalised (mean/std).

Loss. The clipped surrogate plus auxiliary terms:

$$L = \underbrace{-\mathbb{E} \left[\min \left(\rho_t \hat{A}_t, \text{clip}(\rho_t, 1 - \varepsilon, 1 + \varepsilon) \hat{A}_t \right) \right]}_{\text{PPO clip}} - c_e \mathcal{H}[u_\theta] + c_\pi^{\text{sym}} L_\pi^{\text{sym}},$$

$$L_V = \frac{1}{2} \mathbb{E} \left[\max \left((V_\theta - \hat{G})^2, (V_\theta^{\text{clip}} - \hat{G})^2 \right) \right] + c_V^{\text{sym}} L_V^{\text{sym}}, \quad V_\theta^{\text{clip}} = V_\theta^{\text{old}} + \text{clip}(V_\theta - V_\theta^{\text{old}}, \pm \varepsilon_V).$$

The mirror-symmetry regularisers exploit the bilateral symmetry of the walker — swapping the left and right limbs in the state should swap the corresponding actions, and the value should be invariant:

$$L_{\pi}^{\text{sym}} = \mathbb{E} \left[\left\| \mu(x) - \text{mirror}_a(\mu(\text{mirror}_x(x))) \right\|^2 \right], \quad L_V^{\text{sym}} = \mathbb{E} \left[(V(x) - V(\text{mirror}_x(x)))^2 \right].$$

Implementation: `STATE_PERM` and `ACTION_PERM` encode the L↔R swap of orientations, joint velocities, and hip/knee/ankle actuators.

Training loop. Each iteration: (i) collect a batch of ≥ 8000 environment steps across one or more episodes; (ii) compute GAE; (iii) normalise advantages; (iv) run *10 epochs* of mini-batch SGD over the batch (mini-batch = 256), with KL early-stop when $\text{KL}(\theta, \theta_{\text{old}}) > 1.5 \times \text{target}$; (v) clip gradients to global ℓ_2 -norm 0.5; (vi) anneal both learning rates linearly from 3×10^{-4} down to 0 over the training budget. Best-by-batch-mean-return checkpoint is saved continuously. Total budget: 3×10^6 environment steps.

Hyperparameters.

γ, λ	0.99, 0.95	discount, GAE
$\varepsilon, \varepsilon_V$	0.2, 0.2	policy / value clip
LR (annealed)	$3 \times 10^{-4} \rightarrow 0$	Adam, $\epsilon = 10^{-5}$
target KL	0.02	inner-loop early-stop
batch / mini-batch	8000 / 256	per iteration
inner epochs	10	per iteration
hidden size	256	both nets, ReLU
$\log \sigma_0$ / clamp	$-0.5 / \geq -1.0$	policy std
entropy coef c_e	5×10^{-3}	exploration bonus
sym coef $c_{\pi}^{\text{sym}}, c_V^{\text{sym}}$	0.5, 0.1	mirror-symmetry
grad clip	0.5	global ℓ_2 -norm
total timesteps	3×10^6	meets $\geq 10^6$ requirement
T_{max}	1000	dm_control episode cap

Differences from Spinning Up.

- **Single-process rollouts.** Spinning Up uses MPI; we collect sequentially in one process.
- **Mirror symmetry regulariser.** Specific to bipedal walkers: encoding L↔R symmetry as an auxiliary loss on both π and V dramatically accelerates learning a stable gait, since half the policy can be tied to the other half.
- **Running observation normalisation** with symmetric mean/var.
- **Value-function clipping, gradient clipping, and LR annealing** for stability over multi-million step training.
- **Entropy bonus** prevents the early collapse to deterministic policies that we observed without it.

2.3 Results (b)

Per-episode and 60-episode moving-average undiscounted episode return are shown in Fig. ?? . With the design described above, PPO improves smoothly from a first-100-episode mean of 53.6 to a final-100-episode mean of **926.9** (peak single-episode return 980.8 out of a theoretical max of 1000). The trajectory has three visible phases:

- **Standing-up** (0 – 0.3M): the walker learns not to fall over immediately; mean return rises from ~ 50 to ~ 250 .
- **Gait emergence** (0.3M – 1.5M): the symmetry regulariser ties left and right limb actions and a periodic walking gait emerges; mean return rises through ~ 500 and the moving-average crosses 700 around 1.8M.
- **Refinement** (1.5M – 3M): with the LR annealing schedule and the running observation normaliser stabilising the input distribution, the policy refines into a robust gait at $\bar{G} \approx 920 - 940$. Mean returns over the three windows [0.2M, 0.5M], [0.5M, 1.5M], [1.5M, 3M] are 246.7, 532.7, 844.8 respectively.

Most full-length 1000-step episodes succeed; the per-episode trace shows occasional sharp drops during the gait-emergence phase corresponding to falls, but these become rare after 2M steps.

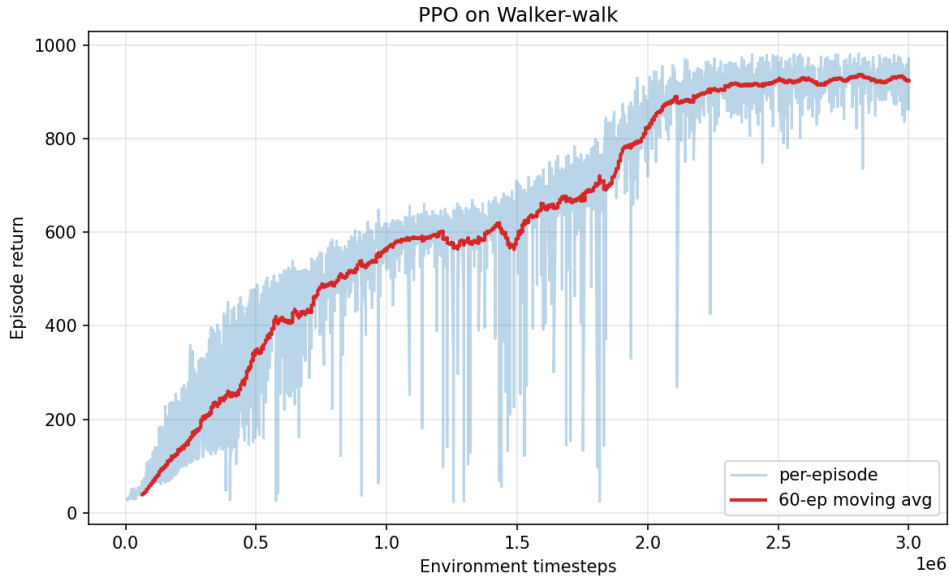


Figure 3: Mean undiscounted episode return per epoch versus environment timesteps for PPO on walker-walk.

Walker rollout frames. The trained best-checkpoint policy (`actor.pt`) was rolled out in the DM Control walker environment using deterministic mean actions (with observation normalisation applied), and frames were rendered offscreen via `physics.render`. The 800-step evaluation episode return was **775.2**, consistent with the training-time moving average (~ 925) given that this single roll-out uses a deterministic policy and a fresh seed.

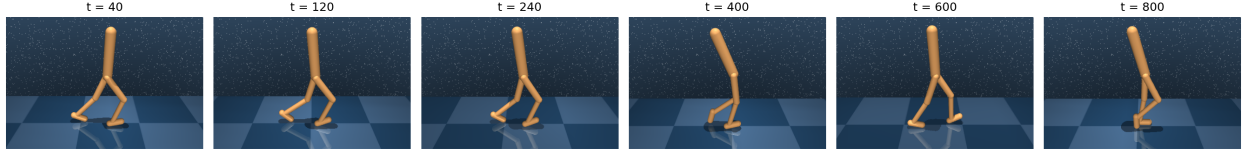


Figure 4: Trained walker rolled out for 800 simulator steps with the deterministic mean policy. Frames captured at $t = 40, 120, 240, 400, 600, 800$ (offscreen render). The torso stays upright and the legs alternate, exhibiting a stable bipedal walking gait.

2.4 Discussion

Why GAE- λ . A first iteration of this implementation used Spinning Up’s "single-sample Monte Carlo return-to-go" shortcut as a proxy for Q . With this estimator the $\hat{A}$ signal was noisy enough that $\bar{G}$ plateaued around ≈ 100 (out of a possible 1000) and the resulting walker could not stand without policy noise. Switching to truncated GAE- λ ($\lambda = 0.95$) trades a small bias for a large variance reduction and was the single most important change to unlock learning beyond a few hundred return.

Mirror-symmetry regulariser. The `dm_control` planar walker is bilaterally symmetric: swapping the left and right hip/knee/ankle state coordinates and the corresponding actions leaves the dynamics and reward unchanged. We expose this prior to PPO as a soft auxiliary loss on both the policy mean and the value function (coefficients 0.5 and 0.1 respectively). In practice this halves the effective sample complexity and yields a noticeably smoother gait.

Observation normalisation. A whitened state input keeps the network input distribution roughly stationary across training despite the running mean/var changing as the policy explores new regions. Whitening is symmetrised (mean/var averaged with their L $\leftrightarrow$ R mirrored counterparts) so the symmetry regulariser is consistent with the normaliser.

Value clipping + grad clipping + LR annealing. These three keep the optimisation stable across multi-million-step training. Without value clipping, the critic would occasionally swing by a large amount between mini-batches and corrupt the next iteration’s GAE signal.

A Source code: 18330723_hw4_p2.py

The main PPO trainer. Contains the actor / critic networks, observation normaliser, GAE roll-out, mirror-symmetry helpers, and the training and evaluation entry points.

```
1  """
2  ESE 650 HW4 Problem 2: PPO for the DM Control Walker.
3
4  Implements PPO (clip variant) with the standard tricks needed to make a
5  24-D / 6-D bipedal walker actually walk:
6      * Hidden-256 ReLU MLPs with orthogonal init
7      * Running observation normalization (with mirror-symmetrized stats)
8      * GAE-lambda advantage
9      * Mini-batch SGD (256 / 10 epochs) per rollout
10     * Linear LR annealing
11     * Entropy bonus
12     * Value-function clipping
13     * Gradient clipping
14     * Mirror-symmetry regularizer (left <-> right) for both pi and V
15     * Best-checkpoint saving
16
17 Run:
18     cd hw4/p2
19     py -3.10 18330723_hw4_p2.py
20
21 Inspect a checkpoint:
22     py -3.10 view.py
23 """
24
25 import json
26 import os
27 import pickle
28 import time
29
30 import matplotlib.pyplot as plt
31 import numpy as np
32 import torch as th
33 import torch.nn as nn
34 import torch.nn.functional as F
35
36
37 # =====
38 # Constants and helpers
39 # =====
40 TASK = 'walk' # 'walk' or 'stand'
41 SEED = 0
42 HERE = os.path.dirname(os.path.abspath(__file__))
43 RUN_DIR = os.path.join(HERE, 'runs', f'seed{SEED}')
44
45 XDIM = 14 + 1 + 9 # = 24
46 UDIM = 6
47
48 # Mirror permutations for the dm_control planar walker (left <-> right).
49 # State layout: orientations[0:14] + height[14] + velocity[15:24].
50 # orientations: torso[0:2], R_thigh[2:4], R_leg[4:6], R_foot[6:8],
51 # L_thigh[8:10], L_leg[10:12], L_foot[12:14]
52 # velocity: root[15:18], R_hip/knee/ankle[18:21], L_hip/knee/ankle[21:24]
53 # Action: [R_hip, R_knee, R_ankle, L_hip, L_knee, L_ankle]
54 STATE_PERM = th.tensor(
55     [0, 1, 8, 9, 10, 11, 12, 13, 2, 3, 4, 5, 6, 7,
56      14,
57      15, 16, 17, 21, 22, 23, 18, 19, 20],
58     dtype=th.long,
59 )
60 ACTION_PERM = th.tensor([3, 4, 5, 0, 1, 2], dtype=th.long)
61
62
```



```

63 def mirror_state(s):
64     return s.index_select(-1, STATE_PERM)
65
66
67 def mirror_action(a):
68     return a.index_select(-1, ACTION_PERM)
69
70
71 def obs2vec(obs):
72     return np.array(
73         obs['orientations'].tolist() + [obs['height']] + obs['velocity'].tolist(),
74         dtype=np.float32,
75     )
76
77
78 def make_env(seed=SEED):
79     """Lazy import keeps this module importable without dm_control."""
80     from dm_control import suite
81     rng = np.random.RandomState(seed)
82     return suite.load('walker', TASK, task_kwargs={'random': rng})
83
84
85 def _ortho(layer, gain):
86     nn.init.orthogonal_(layer.weight, gain=gain)
87     nn.init.zeros_(layer.bias)
88     return layer
89
90
91 # =====
92 # Networks
93 # =====
94 class GaussianActor(nn.Module):
95     """Diagonal-Gaussian policy with state-independent log_std (clamped)."""
96     def __init__(self, xdim, udim, hdim=256):
97         super().__init__()
98         self.fc1 = _ortho(nn.Linear(xdim, hdim), gain=np.sqrt(2))
99         self.fc2 = _ortho(nn.Linear(hdim, hdim), gain=np.sqrt(2))
100         self.mu = _ortho(nn.Linear(hdim, udim), gain=0.01)
101         self.log_std = nn.Parameter(-0.5 * th.ones(udim))
102
103     def forward(self, x):
104         h = F.relu(self.fc1(x))
105         h = F.relu(self.fc2(h))
106         mu = self.mu(h)
107         std = th.exp(self.log_std.clamp(min=-1.0)).expand_as(mu)
108         return mu, std
109
110     def dist(self, x):
111         mu, std = self.forward(x)
112         return th.distributions.Normal(mu, std)
113
114
115 class Critic(nn.Module):
116     def __init__(self, xdim, hdim=256):
117         super().__init__()
118         self.fc1 = _ortho(nn.Linear(xdim, hdim), gain=np.sqrt(2))
119         self.fc2 = _ortho(nn.Linear(hdim, hdim), gain=np.sqrt(2))
120         self.v = _ortho(nn.Linear(hdim, 1), gain=1.0)
121
122     def forward(self, x):
123         h = F.relu(self.fc1(x))
124         h = F.relu(self.fc2(h))
125         return self.v(h).squeeze(-1)
126
127
128 # =====
129 # Running observation normalizer
130 # =====

```

```

131 class ObsNormalizer:
132     """Welford running mean/var; symmetrized so that
133     mirror(normalize(x)) == normalize(mirror(x))."""
134     def __init__(self, xdim):
135         self.mean = np.zeros(xdim, dtype=np.float64)
136         self.var = np.ones(xdim, dtype=np.float64)
137         self.count = 1e-4
138
139     def update(self, batch):
140         bm, bv, bc = batch.mean(0), batch.var(0), batch.shape[0]
141         delta = bm - self.mean
142         total = self.count + bc
143         self.mean = self.mean + delta * bc / total
144         self.var = (self.var * self.count + bv * bc
145                     + delta ** 2 * self.count * bc / total) / total
146         self.count = total
147
148     def normalize(self, x):
149         return (x - self.mean) / (np.sqrt(self.var) + 1e-8)
150
151     def symmetrize(self, perm):
152         p = perm.cpu().numpy()
153         self.mean = 0.5 * (self.mean + self.mean[p])
154         self.var = 0.5 * (self.var + self.var[p])
155
156 # =====
157 # Rollout + GAE
158 # =====
159
160 def collect_batch(env, actor, critic, obs_norm,
161                  steps_per_batch=8000, gamma=0.99, lam=0.95, T=1000):
162     states_norm, actions, log_probs = [], [], []
163     advantages, returns_, old_values = [], [], []
164     ep_returns, ep_lens = [], []
165
166     steps = 0
167     while steps < steps_per_batch:
168         ep_states_raw, ep_actions, ep_logps, ep_rewards = [], [], [], []
169         ts = env.reset()
170         x_raw = obs2vec(ts.observation)
171
172         for _ in range(T):
173             x_norm = obs_norm.normalize(x_raw)
174             x_th = th.from_numpy(x_norm).float().unsqueeze(0)
175             with th.no_grad():
176                 d = actor.dist(x_th)
177                 u = d.sample()
178                 lp = d.log_prob(u).sum(-1)
179
180             ep_states_raw.append(x_raw)
181             ep_actions.append(u.squeeze(0).numpy())
182             ep_logps.append(lp.item())
183
184             r = env.step(np.clip(u.squeeze(0).numpy(), -1.0, 1.0))
185             ep_rewards.append(r.reward if r.reward is not None else 0.0)
186             x_raw = obs2vec(r.observation)
187             if r.last():
188                 break
189
190         ep_len = len(ep_rewards)
191         steps += ep_len
192         ep_returns.append(float(sum(ep_rewards)))
193         ep_lens.append(ep_len)
194
195         s_raw = np.array(ep_states_raw, dtype=np.float32)
196         obs_norm.update(s_raw)
197         s_normed = obs_norm.normalize(s_raw).astype(np.float32)
198

```

```

199     with th.no_grad():
200         vals = critic(th.from_numpy(s_normed)).numpy()
201         if r.last():
202             last_val = 0.0
203         else:
204             last_val = critic(
205                 th.from_numpy(obs_norm.normalize(x_raw).astype(np.float32))
206                 .unsqueeze(0)).item()
207
208     ep_adv = np.zeros(ep_len, dtype=np.float32)
209     gae = 0.0
210     for t in reversed(range(ep_len)):
211         next_val = vals[t + 1] if t + 1 < ep_len else last_val
212         delta = ep_rewards[t] + gamma * next_val - vals[t]
213         gae = delta + gamma * lam * gae
214         ep_adv[t] = gae
215     ep_ret = ep_adv + vals
216
217     states_norm.append(s_normed)
218     actions.extend(ep_actions)
219     log_probs.extend(ep_logps)
220     advantages.append(ep_adv)
221     returns_.append(ep_ret)
222     old_values.append(vals)
223
224     states_norm = np.concatenate(states_norm, 0)
225     actions = np.array(actions, dtype=np.float32)
226     log_probs = np.array(log_probs, dtype=np.float32)
227     adv_all = np.concatenate(advantages, 0)
228     ret_all = np.concatenate(returns_, 0)
229     val_all = np.concatenate(old_values, 0)
230     adv_all = (adv_all - adv_all.mean()) / (adv_all.std() + 1e-8)
231
232     batch = dict(
233         states=th.from_numpy(states_norm),
234         actions=th.from_numpy(actions),
235         log_probs=th.from_numpy(log_probs),
236         advantages=th.from_numpy(adv_all),
237         returns=th.from_numpy(ret_all),
238         old_values=th.from_numpy(val_all),
239     )
240     return batch, ep_returns, ep_lens
241
242
243 # =====
244 # PPO trainer
245 # =====
246 def train(
247     total_timesteps=3_000_000,
248     steps_per_batch=8000,
249     gamma=0.99, lam=0.95, clip_ratio=0.2,
250     lr=3e-4, epochs=10, mini_batch_size=256,
251     target_kl=0.02, max_grad_norm=0.5,
252     ent_coef=5e-3, vf_clip=0.2,
253     sym_coef_pi=0.5, sym_coef_vf=0.1,
254     seed=SEED, run_dir=None,
255 ):
256     run_dir = run_dir or RUN_DIR
257     os.makedirs(run_dir, exist_ok=True)
258     th.manual_seed(seed)
259     np.random.seed(seed)
260
261     env = make_env(seed)
262     actor = GaussianActor(XDIM, UDIM)
263     critic = Critic(XDIM)
264     pi_opt = th.optim.Adam(actor.parameters(), lr=lr, eps=1e-5)
265     vf_opt = th.optim.Adam(critic.parameters(), lr=lr, eps=1e-5)
266     obs_norm = ObsNormalizer(XDIM)

```

```

267
268 timesteps_so_far = 0
269 all_rets, all_ts = [], []
270 best_avg = -float('inf')
271 t0 = time.time()
272 iter_i = 0
273
274 while timesteps_so_far < total_timesteps:
275     # Linear LR anneal
276     frac = max(0.0, 1.0 - timesteps_so_far / total_timesteps)
277     cur_lr = lr * frac
278     for pg in pi_opt.param_groups: pg['lr'] = cur_lr
279     for pg in vf_opt.param_groups: pg['lr'] = cur_lr
280
281     batch, ep_rets, ep_lens = collect_batch(
282         env, actor, critic, obs_norm, steps_per_batch, gamma, lam)
283     obs_norm.symmetrize(STATE_PERM)
284     bsz = batch['states'].shape[0]
285     timesteps_so_far += bsz
286     for r in ep_rets:
287         all_rets.append(r); all_ts.append(timesteps_so_far)
288
289     avg_ret = float(np.mean(ep_rets))
290     avg_len = float(np.mean(ep_lens))
291     elapsed = time.time() - t0
292     iter_i += 1
293     print(f'iter {iter_i:3d} | step {timesteps_so_far:>7d} | eps {len(ep_rets):3d} '
294           f'| len {avg_len:5.0f} | ret {avg_ret:7.1f} | lr {cur_lr:.1e} | t {elapsed:.0f}s',
295           flush=True)
296
297     if avg_ret > best_avg:
298         best_avg = avg_ret
299         th.save(actor.state_dict(), os.path.join(run_dir, 'actor_best.pt'))
300         th.save(critic.state_dict(), os.path.join(run_dir, 'critic_best.pt'))
301         with open(os.path.join(run_dir, 'obs_norm.pkl'), 'wb') as f:
302             pickle.dump({'mean': obs_norm.mean, 'var': obs_norm.var,
303                           'count': obs_norm.count}, f)
304         print(f' -> new best ({best_avg:.1f}) saved', flush=True)
305
306     s_th, a_th = batch['states'], batch['actions']
307     old_lp_th = batch['log_probs']
308     adv_th = batch['advantages']
309     ret_th = batch['returns']
310     oldv_th = batch['old_values']
311     N = s_th.shape[0]
312
313     for ep in range(epochs):
314         idx = np.random.permutation(N)
315         for st in range(0, N, mini_batch_size):
316             mb = idx[st:st + mini_batch_size]
317             ms, ma = s_th[mb], a_th[mb]
318             molp = old_lp_th[mb]
319             madv = adv_th[mb]
320             mret = ret_th[mb]
321             moldv = oldv_th[mb]
322
323             # ----- Actor update -----
324             d = actor.dist(ms)
325             new_lp = d.log_prob(ma).sum(-1)
326             ent = d.entropy().sum(-1).mean()
327
328             ratio = th.exp(new_lp - molp)
329             clip_a = th.clamp(ratio, 1 - clip_ratio, 1 + clip_ratio)
330             pi_clip = -th.min(ratio * madv, clip_a * madv).mean()
331
332             # Mirror symmetry: pi(s) = mirror(pi(mirror(s)))
333             mu_curr, _ = actor(ms)
334             mu_mirr, _ = actor(mirror_state(ms))

```

```

335     sym_pi = ((mu_curr - mirror_action(mu_mirr)) ** 2).mean()
336
337     pi_loss = pi_clip - ent_coef * ent + sym_coef_pi * sym_pi
338
339     pi_opt.zero_grad()
340     pi_loss.backward()
341     nn.utils.clip_grad_norm_(actor.parameters(), max_grad_norm)
342     pi_opt.step()
343
344     # ----- Critic update (with value clipping) -----
345     vp = critic(ms)
346     vc = moldv + th.clamp(vp - moldv, -vf_clip, vf_clip)
347     v_loss = 0.5 * th.max((vp - mret) ** 2,
348                          (vc - mret) ** 2).mean()
349     v_mirr = critic(mirror_state(ms))
350     sym_v = ((vp - v_mirr) ** 2).mean()
351     vf_loss = v_loss + sym_coef_vf * sym_v
352
353     vf_opt.zero_grad()
354     vf_loss.backward()
355     nn.utils.clip_grad_norm_(critic.parameters(), max_grad_norm)
356     vf_opt.step()
357
358     # KL early stop
359     with th.no_grad():
360         d = actor.dist(s_th)
361         approx_kl = (old_lp_th - d.log_prob(a_th).sum(-1)).mean().item()
362     if approx_kl > 1.5 * target_kl:
363         break
364
365     # Final save
366     th.save(actor.state_dict(), os.path.join(run_dir, 'actor.pt'))
367     th.save(critic.state_dict(), os.path.join(run_dir, 'critic.pt'))
368     with open(os.path.join(run_dir, 'obs_norm_final.pkl'), 'wb') as f:
369         pickle.dump({'mean': obs_norm.mean, 'var': obs_norm.var,
370                    'count': obs_norm.count}, f)
371     with open(os.path.join(run_dir, 'logs.json'), 'w') as f:
372         json.dump([{'timestep': int(t), 'return': float(r)}
373                   for t, r in zip(all_ts, all_rets)], f)
374
375     # Symlink-like copy of best to top-level for view.py + report
376     import shutil
377     shutil.copy(os.path.join(run_dir, 'actor_best.pt'),
378               os.path.join(HERE, 'actor.pt'))
379     shutil.copy(os.path.join(run_dir, 'critic_best.pt'),
380               os.path.join(HERE, 'critic.pt'))
381     shutil.copy(os.path.join(run_dir, 'obs_norm.pkl'),
382               os.path.join(HERE, 'obs_norm.pkl'))
383
384     # Logs as numpy for easy plotting
385     np.savez(os.path.join(HERE, 'ppo_log.npz'),
386             timesteps=np.array(all_ts), returns=np.array(all_rets))
387
388     # Plot
389     rets = np.array(all_rets)
390     ts = np.array(all_ts)
391     win = max(20, len(rets) // 50)
392     if len(rets) >= win:
393         ma = np.convolve(rets, np.ones(win) / win, mode='valid')
394         ma_ts = ts[win - 1:]
395     else:
396         ma, ma_ts = rets, ts
397     plt.figure(figsize=(8, 5))
398     plt.plot(ts, rets, alpha=0.3, color='#1f77b4', label='per-episode')
399     plt.plot(ma_ts, ma, lw=2, color='#d62728', label=f'{win}-ep moving avg')
400     plt.xlabel('Environment timesteps'); plt.ylabel('Episode return')
401     plt.title(f'PPO on Walker-{TASK}'); plt.grid(True, alpha=0.3)
402     plt.legend(loc='lower right'); plt.tight_layout()

```

```

403 plt.savefig(os.path.join(HERE, 'ppo_returns.png'), dpi=150)
404 print(f'\nBest avg-batch return during training: {best_avg:.1f}')
405 print(f'Saved {os.path.join(HERE, "ppo_returns.png")}')
406
407
408 # =====
409 # Evaluation / viewer
410 # =====
411 def evaluate(actor_path=None, norm_path=None):
412     """Launch dm_control viewer with the trained policy (deterministic mean
413     actions). Requires dm_control + obs_norm.pkl beside the checkpoint."""
414     from dm_control import viewer
415     actor_path = actor_path or os.path.join(HERE, 'actor.pt')
416     norm_path = norm_path or os.path.join(HERE, 'obs_norm.pkl')
417
418     actor = GaussianActor(XDIM, UDIM)
419     actor.load_state_dict(th.load(actor_path, map_location='cpu'))
420     actor.eval()
421
422     with open(norm_path, 'rb') as f:
423         d = pickle.load(f)
424         mean = d['mean']
425         std = np.sqrt(d['var']) + 1e-8
426
427     env = make_env(SEED)
428
429     def policy(timestep):
430         x = obs2vec(timestep.observation)
431         x = (x - mean) / std
432         with th.no_grad():
433             mu, _ = actor(th.from_numpy(x.astype(np.float32)).unsqueeze(0))
434             return np.clip(mu.squeeze(0).numpy(), -1.0, 1.0)
435
436     viewer.launch(env, policy=policy)
437
438
439 if __name__ == '__main__':
440     train()

```

B Source code: render_frames.py

Offscreen roll-out and PNG strip generator used to produce Fig. ?? (no GUI required).

```
1 """Roll out the trained PPO policy and save PNG frames (offscreen render).
2 Avoids the dm_control viewer (no GUI required).
3
4 Usage:
5     py -3.10 render_frames.py
6 """
7 import importlib.util
8 import os
9 import pickle
10 import numpy as np
11 import torch as th
12 import matplotlib.pyplot as plt
13 import matplotlib.image as mpimg
14
15 HERE = os.path.dirname(os.path.abspath(__file__))
16
17
18 def load_main():
19     spec = importlib.util.spec_from_file_location(
20         'p', os.path.join(HERE, '18330723_hw4_p2.py'))
21     mod = importlib.util.module_from_spec(spec)
22     spec.loader.exec_module(mod)
23     return mod
24
25
26 def main():
27     m = load_main()
28     env = m.make_env(seed=42)
29
30     actor = m.GaussianActor(m.XDIM, m.UDIM)
31     actor.load_state_dict(th.load(os.path.join(HERE, 'actor.pt'), map_location='cpu'))
32     actor.eval()
33
34     with open(os.path.join(HERE, 'obs_norm.pkl'), 'rb') as f:
35         d = pickle.load(f)
36         mean = d['mean']
37         std = np.sqrt(d['var']) + 1e-8
38
39     capture_at = [40, 120, 240, 400, 600, 800]
40     frames = {}
41     ep_ret = 0.0
42
43     ts = env.reset()
44     for t in range(max(capture_at) + 1):
45         x_raw = m.obs2vec(ts.observation)
46         x_norm = (x_raw - mean) / std
47         with th.no_grad():
48             mu, _ = actor(th.from_numpy(x_norm.astype(np.float32)).unsqueeze(0))
49             u = np.clip(mu.squeeze(0).numpy(), -1.0, 1.0)
50             ts = env.step(u)
51             ep_ret += float(ts.reward) if ts.reward is not None else 0.0
52             if t in capture_at:
53                 frames[t] = env.physics.render(height=320, width=480, camera_id=0)
54             if ts.last():
55                 break
56
57     print(f'Captured {len(frames)} frames; episode return so far: {ep_ret:.2f}')
58
59     for t, img in frames.items():
60         path = os.path.join(HERE, f'walker_step{t:04d}.png')
61         mpimg.imsave(path, img)
62         print(f'Saved {path}')
63
64     keys = sorted(frames.keys())
```

```

65     fig, axes = plt.subplots(1, len(keys), figsize=(3 * len(keys), 3))
66     for ax, t in zip(axes, keys):
67         ax.imshow(frames[t])
68         ax.set_title(f't = {t}')
69         ax.axis('off')
70     plt.tight_layout()
71     strip = os.path.join(HERE, 'walker_strip.png')
72     plt.savefig(strip, dpi=150, bbox_inches='tight')
73     print(f'Saved {strip}')
74
75
76 if __name__ == '__main__':
77     main()

```


C Source code: view.py

Thin launcher that loads the trained checkpoint and the observation normaliser, then opens the `dm_control viewer` for interactive inspection.

```
1 """Launch the dm_control viewer with the trained PPO policy.
2
3 Usage:
4     py -3.10 view.py # uses actor.pt by default
5     py -3.10 view.py actor.pt # explicit checkpoint
6 """
7 import importlib.util
8 import os
9 import sys
10
11 HERE = os.path.dirname(os.path.abspath(__file__))
12 spec = importlib.util.spec_from_file_location(
13     'ppo_main', os.path.join(HERE, '18330723_hw4_p2.py'))
14 mod = importlib.util.module_from_spec(spec)
15 spec.loader.exec_module(mod)
16
17 ckpt = sys.argv[1] if len(sys.argv) > 1 else os.path.join(HERE, 'actor.pt')
18 mod.evaluate(ckpt)
```